HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG — KHOA AN TOÀN THÔNG TIN

Đánh giá an toàn và bảo vệ dữ liệu người dùng ứng dụng OWASP Juice Shop v20.2.0

Học phần: DevOps và DevSecOps (SEC4349)

Thành viên	MSSV	Lỗi phụ trách
Nguyễn Toàn Thắng	B26CHAT039	F01 — SQL injection đăng nhập
Phạm Thị Minh Thúy	B26CHAT042	F02 — IDOR giỏ hàng
Đỗ Đức Quốc Anh	B26CHAT001	F03 — lộ endpoint /metrics
Hồ Anh Vũ	B26CHAT048	F04 — cấu hình an toàn container

Năm học 2026

Mục lục

Danh mục từ viết tắt

1. Giới thiệu và phạm vi

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

2.1. Nghiệp vụ đăng ký và đăng nhập

2.2. Nghiệp vụ giỏ hàng và thanh toán

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

3.1. Công cụ sử dụng

3.2. Chính sách và tiêu chuẩn đối chiếu

4. Quy trình DevSecOps và các cổng kiểm soát

4.1. Quét và phát hiện lỗi — kết quả thực tế

5. Bốn lỗi an toàn: phát hiện và xử lý

5.1. F01 — SQL injection tại đăng nhập

5.2. F02 — IDOR khi đọc giỏ hàng

5.3. F03 — Lộ endpoint /metrics không xác thực

5.4. F04 — Sai cấu hình an toàn của container

6. Kết luận

7. Tài liệu tham khảo

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Phụ lục C — Giải thích tiêu chuẩn và điều khoản tham chiếu

Danh mục từ viết tắt

Bảng 1. Danh mục từ viết tắt

Viết tắt	Giải nghĩa
SQLi	SQL Injection — tiêm nhiễm câu lệnh SQL
IDOR	Insecure Direct Object Reference — tham chiếu đối tượng thiếu kiểm soát truy cập
JWT	JSON Web Token — token phiên
SAST	Phân tích tĩnh mã nguồn (Static Application Security Testing)
CI/CD	Tích hợp / triển khai liên tục
OPA	Open Policy Agent (ngôn ngữ Rego, công cụ Conftest)
GDPR	Quy định bảo vệ dữ liệu của EU (2016/679)
PCI-DSS	Chuẩn an toàn dữ liệu thẻ thanh toán
CIS	Center for Internet Security (CIS Docker Benchmark)
CWE / OWASP	Phân loại điểm yếu phần mềm / tổ chức an toàn ứng dụng web
UID	Mã định danh người dùng hệ điều hành

1. Giới thiệu và phạm vi

Báo cáo đánh giá an toàn ứng dụng **OWASP Juice Shop v20.2.0** dưới góc độ bảo vệ dữ liệu người dùng. Bốn thành viên thực hiện, mỗi người chịu trách nhiệm một lỗi an toàn: hai lỗi nằm ở mã nguồn, hai lỗi ở khâu triển khai và vận hành. Các lỗi đều được trình bày theo mạch **trước khi sửa → sau khi sửa**, lần lượt gồm cách phát hiện, cách xử lý, chính sách bị vi phạm, pha DevSecOps tương ứng và đoạn mã demo.

Mỗi kịch bản được chạy song song trên hai môi trường để đối chiếu: bản **trước khi sửa** là container chạy image gốc (127.0.0.1:3100), còn bản **sau khi sửa** là bản triển khai đã qua reverse proxy (127.0.0.1:3102). Bên trong container, cổng 3000 là cổng mặc định của Juice Shop; bản đã xử lý vẫn lắng nghe cổng này nhưng không phơi ra host, chỉ reverse proxy (cổng 3102) truy cập được qua mạng nội bộ. Cổng 3100 dành cho container đối chứng chạy image gốc để so sánh. Cũng cần nhắc lại rằng Juice Shop cố ý chứa lỗ hổng phục vụ huấn luyện, nên các phát hiện ở đây minh họa quy trình chứ không nhằm đánh giá dự án gốc.

Bảng 2. Phân công thành viên

Thành viên	Lỗi phụ trách	Nhóm lỗi	Pha DevSecOps
Nguyễn Toàn Thắng — B26CHAT039	F01 SQL injection tại đăng nhập	Mã nguồn	Code, Test
Phạm Thị Minh Thúy — B26CHAT042	F02 IDOR khi đọc giỏ hàng	Mã nguồn (phân quyền)	Design, Code, Test
Đỗ Đức Quốc Anh — B26CHAT001	F03 Lộ endpoint /metrics không xác thực	Triển khai/vận hành	Deploy, Operate
Hồ Anh Vũ — B26CHAT048	F04 Sai cấu hình an toàn của container	Triển khai (build)	Build, Release

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

Juice Shop là một ứng dụng web thương mại điện tử mô phỏng (bán nước ép) do OWASP phát hành nhằm huấn luyện an toàn ứng dụng. Về kiến trúc, đây là một Single Page Application gồm các thành phần sau:

Bảng 3. Thành phần của ứng dụng Juice Shop

Thành phần	Công nghệ	Vai trò
Giao diện người dùng	Angular (SPA)	Hiển thị, gọi REST API
Máy chủ ứng dụng	Node.js + Express	Xử lý REST API và nghiệp vụ
Truy xuất dữ liệu	ORM Sequelize trên SQLite	Lưu Users, Baskets, Cards…
Xác thực phiên	JWT (RS256), thư viện jsonwebtoken	Cấp và kiểm tra token đăng nhập
Giám sát	prom-client, endpoint `/metrics`	Xuất chỉ số dạng Prometheus
Đóng gói	Docker image distroless, chạy UID 65532	Triển khai bằng container

Cả bốn lỗi được phân tích đều gắn với những thành phần này: REST API và ORM (F01, F02), endpoint giám sát (F03) cùng cách đóng gói container (F04). Yêu cầu bảo vệ dữ liệu được xác định theo GDPR (nguyên tắc ở Điều 5, nghĩa vụ an toàn ở Điều 32) và PCI-DSS cho dữ liệu thẻ thanh toán. Song song với khung quốc tế, báo cáo còn đối chiếu pháp luật Việt Nam, gồm **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân và **Luật An toàn thông tin mạng 2015** (chi tiết ở mục 3.2).

Bảng 4. Dữ liệu cần bảo vệ và cơ sở chính sách

Dữ liệu	Yêu cầu bảo vệ	Cơ sở chính sách
Email, hồ sơ	Chỉ chủ tài khoản/quản trị truy cập	GDPR Điều 5(1)(f), 25(2)
Mật khẩu	Không để lộ; không bị vượt xác thực	GDPR Điều 32(1)(b)
Token phiên (JWT)	Chỉ cấp khi xác thực đúng	GDPR Điều 32(1)(b)
Giỏ hàng	Ràng buộc chủ sở hữu ở mọi truy vấn	GDPR Điều 5(1)(f), 25(2)
Thẻ thanh toán	Không lưu/không lộ số thẻ đầy đủ	PCI-DSS 3.4; GDPR Điều 32

Ý nghĩa cụ thể của từng điều khoản (GDPR, PCI-DSS… và các chuẩn khác) được tóm gọn trong **Phụ lục C**.

2.1. Nghiệp vụ đăng ký và đăng nhập

Người dùng đăng ký qua `POST /api/Users` và đăng nhập qua `POST /rest/user/login`. Route `routes/login.ts` truy vấn bảng Users, nếu khớp thì cấp JWT kèm mã giỏ hàng. Vấn đề nằm ở ranh giới tin cậy: email do client gửi lên đi thẳng vào câu lệnh SQL, và đó chính là gốc của lỗi **F01**.

2.2. Nghiệp vụ giỏ hàng và thanh toán

Sau khi đăng nhập, người dùng gọi `GET /rest/basket/:id` để xem giỏ. Route `routes/basket.ts` tìm giỏ theo `id` trên URL nhưng bỏ qua việc kiểm tra giỏ đó có thuộc về người gọi hay không, và đây là gốc của lỗi **F02**. Vì dữ liệu giỏ hàng phản ánh thói quen mua sắm, nó cần được bảo vệ chặt theo quyền sở hữu.

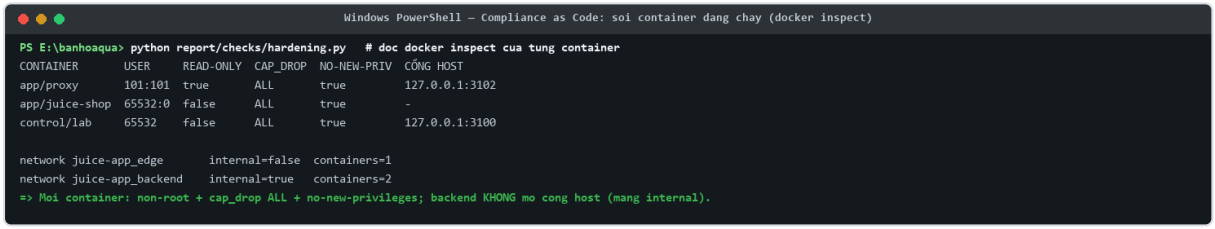
3. Môi trường triển khai và yêu cầu an toàn tối thiểu

Ứng dụng chạy bằng Docker trên Windows (WSL2) và được triển khai qua `docker-compose.yml`. Các yêu cầu an toàn tối thiểu ở đây bám theo CIS Docker Benchmark cùng nguyên tắc giảm bề mặt tấn công.

Bảng 5. Kiểm soát an toàn tối thiểu

Kiểm soát	Cách thực hiện	Tiêu chuẩn
Giảm bề mặt mạng	Chỉ reverse proxy publish cổng, chỉ trên 127.0.0.1; backend ở mạng internal	GDPR Điều 32
Không chạy root	Juice Shop UID 65532, Nginx UID 101	CIS Docker 4.1
Đặc quyền tối thiểu	`cap_drop: ALL`, `no-new-privileges`, filesystem chỉ đọc nơi được	CIS Docker 5.3/5.12
Ghim phiên bản	Luật Rego chặn `:latest`, cảnh báo khi base image chưa ghim digest	CIS Docker 4.5
Lọc đường vào	Nginx chặn /metrics, giới hạn 5 lần đăng nhập/phút	GDPR Điều 32

Những kiểm soát này được **xác minh ngay trên container đang chạy** bằng `docker inspect` (Compliance as Code), cho thấy chúng thực sự có hiệu lực chứ không chỉ nằm trên file cấu hình:



Hình 1. Bằng chứng runtime qua `docker inspect`: cả ba container chạy non-root (USER), `cap_drop: ALL`, `no-new-privileges`; backend không mở cổng host (CỔNG HOST = "-", mạng internal), chỉ proxy mở loopback 127.0.0.1:3102

Vẫn còn một số giới hạn: container đối chứng 3100 chạy image gốc chỉ phục vụ so sánh trước/sau; HTTP nội bộ chưa bật TLS; base image chưa ghim theo digest (luật Rego hiện mới cảnh báo chứ chưa bắt buộc trên image đối chứng).

3.1. Công cụ sử dụng

Bảng 6. Công cụ và vai trò

Công cụ	Loại	Vai trò trong báo cáo
Docker, Docker Compose	Nền tảng container	Triển khai ứng dụng và môi trường kiểm thử
Nginx	Reverse proxy	Chặn /metrics, giới hạn tốc độ đăng nhập (F03)
Semgrep	SAST (phân tích tĩnh)	Đọc mã nguồn (không chạy), chỉ ra dòng lỗi F01/F02 — bước *tìm ra* lỗi
Gitleaks	Quét secret	Tìm khóa/bí mật lộ trong mã nguồn
node:test (Supertest)	Kiểm thử động	Chạy ứng dụng thật, tái hiện lỗi *trước khi vá* (đỏ) và kiểm chứng *sau khi vá* (xanh)
Conftest / OPA (Rego)	Policy as Code	Kiểm tra Dockerfile và cấu hình triển khai (F03/F04)

Giới hạn của phân tích tĩnh. SAST (Semgrep) chỉ *tìm ra mẫu lỗi* trên mã nguồn và **không chứng minh được hành vi khi chạy**. Chẳng hạn, nó không biết một endpoint xoá dữ liệu có thực sự xoá hay chỉ ghi nhận yêu cầu, cũng không biết một API có trả về đầy đủ số thẻ hay không. Những điều đó phải nhờ **kiểm thử động** (node:test) và **rà soát của con người** khẳng định. Chính vì thế, báo cáo kết hợp bốn lớp gồm SAST, kiểm thử động, Policy as Code (build-time) và Compliance as Code (runtime), thay vì trông cậy vào một công cụ duy nhất.

3.2. Chính sách và tiêu chuẩn đối chiếu

Các phát hiện được đối chiếu với những chính sách và tiêu chuẩn phổ biến chứ không phải quy ước tự đặt ra, sử dụng **song song** khung quốc tế lẫn pháp luật Việt Nam về bảo vệ dữ liệu cá nhân.

Quốc tế:

- **GDPR** (Regulation (EU) 2016/679): khung bảo vệ dữ liệu cá nhân; dùng các Điều 5 (nguyên tắc), 25 (bảo vệ ngay từ thiết kế) và 32 (an toàn khi xử lý) làm cơ sở đánh giá dữ liệu cần bảo vệ.
- **PCI-DSS v3.2.1**: chuẩn an toàn dữ liệu thẻ thanh toán; Yêu cầu 3.4 (không lưu/không lộ số thẻ đầy đủ), 8.2.1 (không lưu credential ở dạng đọc được), 2.2 (cấu hình an toàn, đặc quyền tối thiểu). *Báo cáo dùng nhất quán đánh số theo v3.2.1* — ở PCI-DSS v4.0 các số này thay đổi (ví dụ 3.4 → 3.5.1).

- **OWASP Top 10 2021 & CWE**: phân loại loại lỗi — A03 Injection/CWE-89 (F01), A01 Broken Access Control/CWE-639 (F02).
- **CIS Docker Benchmark**: chuẩn cấu hình container an toàn — các mục 4.1, 4.5, 4.9, 4.10, 5.8 dùng cho F04.
- **SLSA / OCI image-spec / NIST SP 800-190**: nguồn gốc, toàn vẹn image và an toàn container (F04).

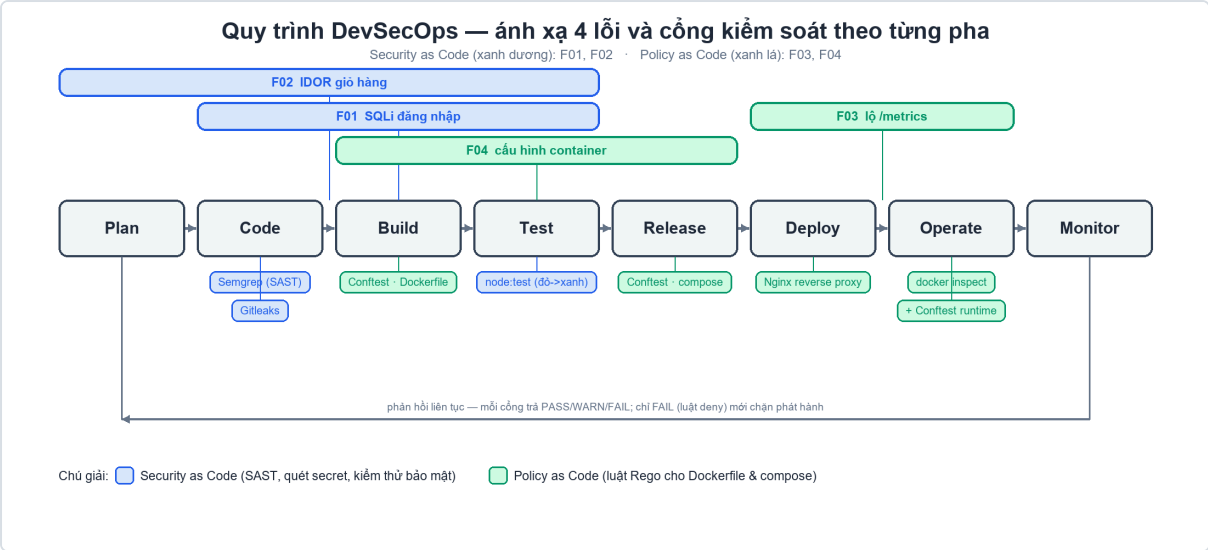
Việt Nam:

- **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân (hiệu lực 01/7/2023): **Điều 3** — 8 nguyên tắc (trong đó có nguyên tắc *bảo mật, an toàn*); **Điều 26** — biện pháp bảo vệ dữ liệu cá nhân gồm biện pháp *quản lý* và *kỹ thuật*, áp dụng ngay từ đầu và suốt quá trình xử lý; **Điều 27** — bảo vệ dữ liệu cá nhân cơ bản. Tương ứng GDPR Điều 5/25/32.
- **Luật An toàn thông tin mạng 2015 (86/2015/QH13)**: **Điều 16** — nguyên tắc bảo vệ thông tin cá nhân; **Điều 17** — không cung cấp/chia sẻ thông tin cá nhân cho bên thứ ba khi chưa được đồng ý (liên quan F02/F03); **Điều 19** — bảo đảm an toàn thông tin cá nhân trên mạng bằng biện pháp quản lý, kỹ thuật và *tuân thủ tiêu chuẩn, quy chuẩn kỹ thuật*.
- **TCVN 11930:2017** — tiêu chuẩn kỹ thuật quốc gia *Yêu cầu cơ bản về an toàn hệ thống thông tin theo cấp độ*, làm cơ sở kỹ thuật cho cấu hình triển khai tối thiểu (F03/F04).

Ý nghĩa cụ thể (**yêu cầu gì**) của mỗi điều khoản tham chiếu được tóm gọn trong **Phụ lục C** (bảng tra). Phần thân chỉ ghi mã điều khoản trong các bảng và các mục, còn ý nghĩa thì tra ở Phụ lục C.

4. Quy trình DevSecOps và các cổng kiểm soát

Việc phát hiện và chống tái diễn cả bốn lỗi đều được viết thành mã và chạy tự động dưới dạng một chuỗi kiểm soát (pipeline cục bộ) theo mô hình DevSecOps. Hình dưới ánh xạ từng lỗi và cổng kiểm soát vào các pha trong vòng đời DevSecOps (xanh dương là Security as Code cho F01/F02, xanh lá là Policy as Code cho F03/F04):

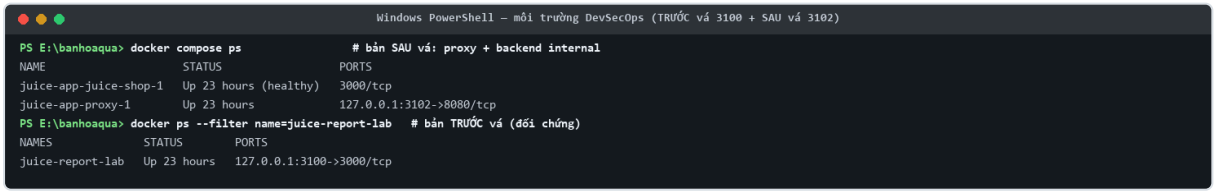


Hình 2. Quy trình DevSecOps: ánh xạ bốn lỗi F01–F04 và các cổng kiểm soát theo từng pha; Security as Code cho F01/F02, Policy as Code cho F03/F04

Hai nhóm kiểm soát trong hình:

- **Security as Code:** quét secret (Gitleaks), SAST (Semgrep) cho F01/F02, và kiểm thử bảo mật `f01-f02.test.ts` (đỏ trước vá, xanh sau vá).
- **Policy as Code (build-time):** luật Rego cho Dockerfile (F04) và cho `docker-compose` (F03), chặn ngay những cấu hình mở cổng backend hoặc chạy root **trước khi triển khai**.
- **Compliance as Code (runtime):** kiểm tra **container đang chạy** bằng `docker inspect` cùng luật Rego runtime (`report/policy/runtime.rego`, `checks/hardening.py`), xác nhận non-root, drop capability và chỉ proxy mở cổng loopback. Việc đặt container đối chứng (trước vá) cạnh bản triển khai (sau vá) tạo thành đúng cặp ca *lỗi cố ý / đạt* của một bài kiểm tra tuân thủ.

Việc kiểm chứng diễn ra trên **hai môi trường chạy song song**: container đối chứng *trước vá* (image gốc, `127.0.0.1:3100`) và bản triển khai *sau vá* (reverse proxy kèm backend ở mạng internal, `127.0.0.1:3102`). Cả hai đều đang hoạt động:



Hình 3. Hai môi trường: đối chứng trước vá (3100) và triển khai sau vá qua proxy (3102); backend chạy ở mạng internal, không mở cổng host

Toàn bộ kiểm soát này được viết thành mã nên chạy lặp lại được; hiện chúng chạy cục bộ nhưng hoàn toàn có thể ghép thành một chuỗi cổng (gate) trong CI. Mỗi cổng trả về

PASS, WARN hoặc FAIL. Chỉ mức **FAIL (luật `deny`)** mới chặn phát hành, còn mức **WARN (luật `warn`)** dừng ở cảnh báo. Cụ thể trong `compose.rego`, các luật `cap_drop` và `read_only` xếp mức WARN (khuyến cáo), trong khi các luật về mở cổng backend, bind ngoài loopback, privileged, mount docker.sock và secret nội tuyến đều ở mức FAIL (chặn).

Bảng 7. Các cổng kiểm soát

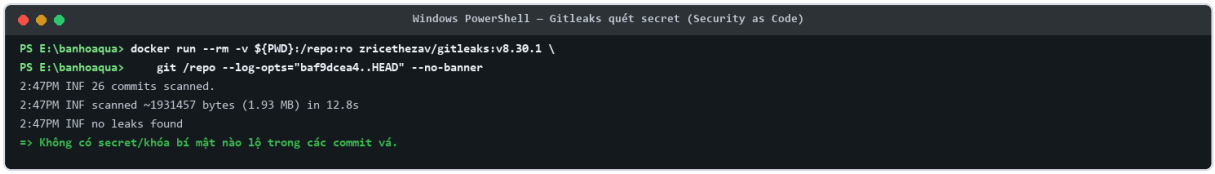
Cổng kiểm soát	Công cụ	Lỗi được kiểm soát	Chặn phát hành
Quét secret	Gitleaks	khóa/bí mật lộ trong mã	có
SAST	Semgrep	F01, F02	có
Kiểm thử bảo mật	node:test (Supertest)	F01, F02 (đỏ→xanh)	có
Policy as Code — build	Conftest/OPA (Rego)	F03, F04 (Dockerfile, compose)	có — luật `deny`; luật `warn` chỉ cảnh báo
Compliance as Code — runtime	Conftest + `docker inspect`	F03, F04 (container đang chạy)	cảnh báo

Báo cáo bám theo **bảy pha của tài liệu lab** (*Hands-On Lab Guide: Security as Code with OWASP Juice Shop & Docker*), chỉ thay một vài công cụ bằng lựa chọn tương đương:

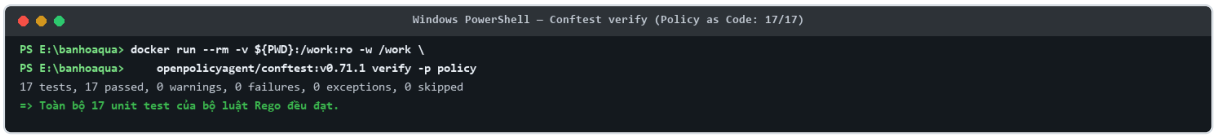
Bảng 8. Bảy pha DevSecOps của tài liệu lab và cách báo cáo phủ

Pha (tài liệu lab)	Hoạt động	Công cụ
1. Thiết lập môi trường	Dựng hai môi trường trước/sau vá bằng Docker	Docker Compose, Node
2. Policy as Code	Kiểm chính sách Dockerfile & compose trước khi triển khai	Conftest/OPA (Rego)
3. SAST	Quét tĩnh tìm điểm yếu trong mã nguồn	Semgrep *(thay SonarQube)*
4. Test-Driven Security	Viết kiểm thử đỏ trước vá → xanh sau vá	node:test (Supertest)
5. Vá ứng dụng	Sửa mã theo kết quả kiểm thử	TypeScript / Express
6. Compliance as Code	Soi container đang chạy (runtime)	Conftest + `docker inspect` *(thay InSpec)*
7. Kiểm chứng	Chạy lại toàn bộ cổng kiểm soát; quét secret	Conftest, Gitleaks

Hai cổng tự động dưới đây đã chạy thật, kèm ảnh chụp output thực tế:



Hình 4. Gitleaks quét toàn bộ commit vá: 24 commit, không có secret/khóa bí mật nào lộ



Hình 5. Conftest verify: toàn bộ 17 unit test của bộ luật Rego (Dockerfile + compose) đều đạt

Đoạn mã demo của mỗi thành viên không nằm ngoài tài liệu môn học mà **áp dụng trực tiếp** kỹ thuật được dạy ở đó:

Bảng 9. Liên hệ đoạn mã demo với tài liệu môn học

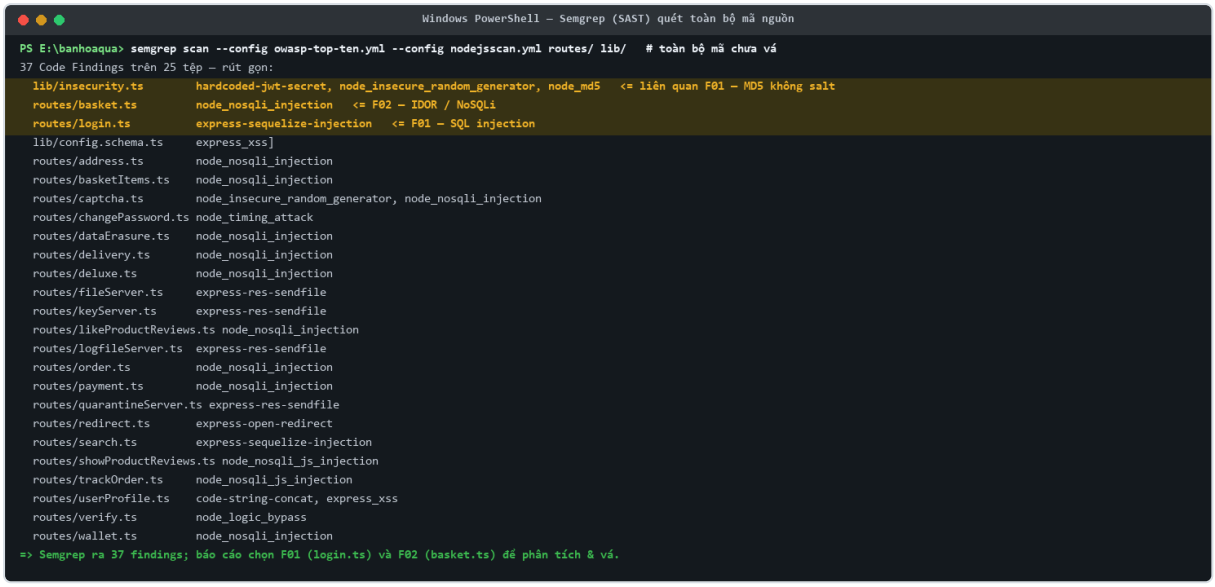
Thành viên	Đoạn mã demo	Kỹ thuật & tài liệu môn học áp dụng
Nguyễn Toàn Thắng (F01)	`demos/tv1_f01_sqli.py`, kiểm thử `f01-f02.test.ts`	Shift-left, SAST, Test-Driven Security — slide kiến trúc DevSecOps
Phạm Thị Minh Thúy (F02)	`demos/tv2_f02_idor.py`, kiểm thử phân quyền	Security as Code, kiểm thử trong CI — slide DevSecOps
Đỗ Đức Quốc Anh (F03)	`demos/tv3_f03_metrics.py`, `nginx.conf`	OpSec giảm phơi nhiễm endpoint, reverse proxy & giới hạn tốc độ — slide OpSec/DevSecOps
Hồ Anh Vũ (F04)	`demos/tv4_f04_policy.py`, `docker_security.rego`	Policy as Code với Conftest — hướng dẫn Policy as Code (Conftest)

Nói cách khác, mỗi thành viên chọn một kỹ thuật từ tài liệu môn học rồi hiện thực hóa nó thành đoạn mã chạy được để phát hiện hoặc chặn lỗi mình phụ trách.

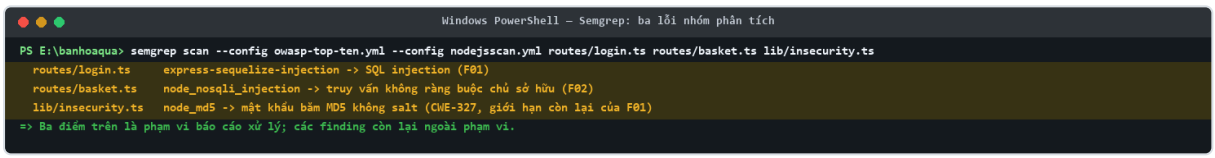
4.1. Quét và phát hiện lỗi — kết quả thực tế

Các cổng quét tự động được chạy trên mã nguồn và cấu hình *chưa vá* để lộ ra điểm yếu trước khi sửa.

SAST (Semgrep). Quét toàn bộ `routes/` và `lib/` cho ra 37 điểm yếu trên 25 tệp (NoSQL injection, XSS, băm yếu, JWT secret cố định…). Trong số đó, báo cáo tập trung ba điểm thuộc phạm vi phân công: F01 ở `login.ts` (SQL injection), F02 ở `basket.ts` (truy vấn giỏ hàng không ràng buộc chủ sở hữu) và hạn chế băm mật khẩu bằng MD5 không salt tại `lib/insecurity.ts`.

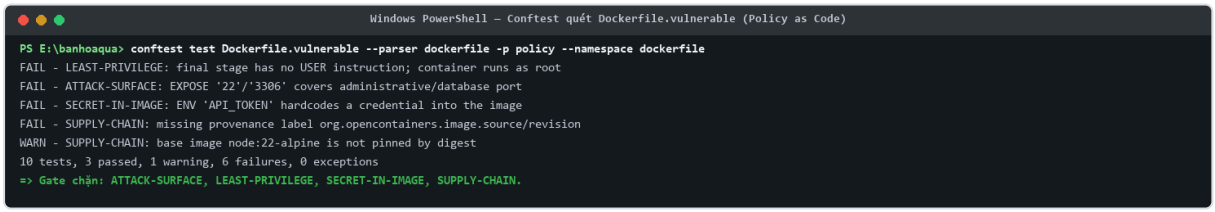


Hình 6. Semgrep quét toàn bộ mã nguồn: 37 điểm yếu trên 25 tệp; tô vàng ba điểm thuộc phạm vi báo cáo



Hình 7. Ba điểm yếu báo cáo xử lý: F01 SQL injection (login.ts), F02 truy vấn giỏ hàng (basket.ts) và MD5 không salt (insecurity.ts)

Policy as Code (Conftest). Quét `Dockerfile.vulnerable` cho ra 6 vi phạm thuộc 4 nhóm luật, đủ để gate chặn phát hành (chi tiết F04 ở mục 5.4):



Hình 8. Conftest quét Dockerfile.vulnerable: 6 vi phạm thuộc 4 nhóm luật, gate chặn phát hành

5. Bốn lỗi an toàn: phát hiện và xử lý

5.1. F01 — SQL injection tại đăng nhập

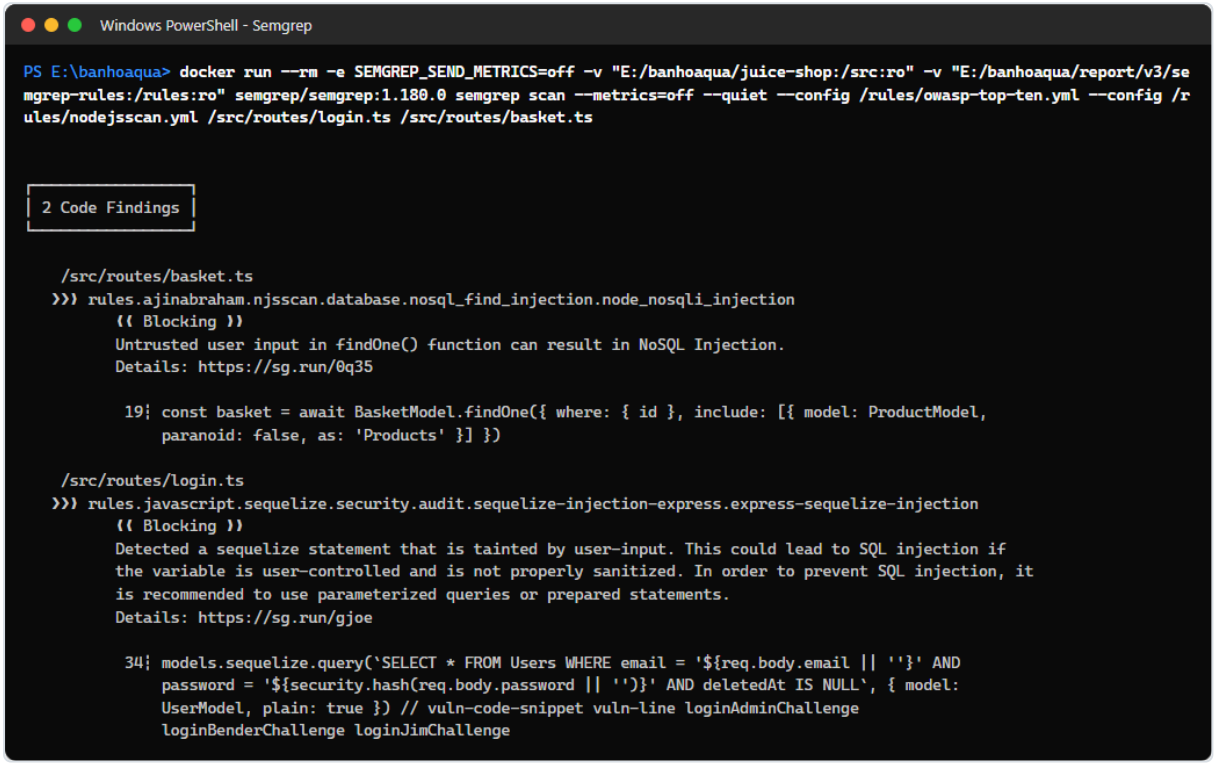
Trước khi sửa

Route `routes/login.ts` ghép thẳng email vào câu SQL:

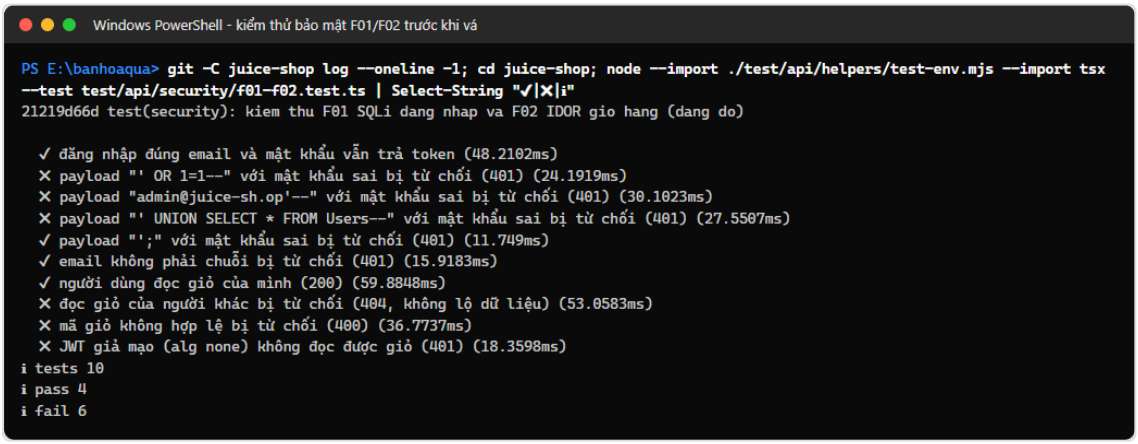
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}'
  AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
UserModel, plain: true })
```

Với email `' OR 1=1--`, mệnh đề WHERE luôn đúng còn phần kiểm tra mật khẩu bị đẩy thành chú thích, nên máy chủ cấp token của tài khoản đầu tiên (tài khoản quản trị) mà không cần mật khẩu. Đây chính là CWE-89, OWASP A03:2021.

Phát hiện. Semgrep (luật `express-sequelize-injection`) chỉ đúng dòng lỗi. Kiểm thử `test/api/security/f01-f02.test.ts` gửi payload kèm mật khẩu sai và đòi nhận về 401; trên mã chưa sửa, test báo đỏ (Hình 10). Demo `demos/tv1_f01_sqli.py` xác nhận token vẫn được cấp (Hình 11).



Hình 9. Semgrep phát hiện truy vấn nhiễm dữ liệu người dùng tại login.ts và basket.ts



Hình 10. Kiểm thử bảo mật F01/F02 trên mã chưa sửa: các ca kiểm thử đỏ ứng với lỗ hổng còn tồn tại

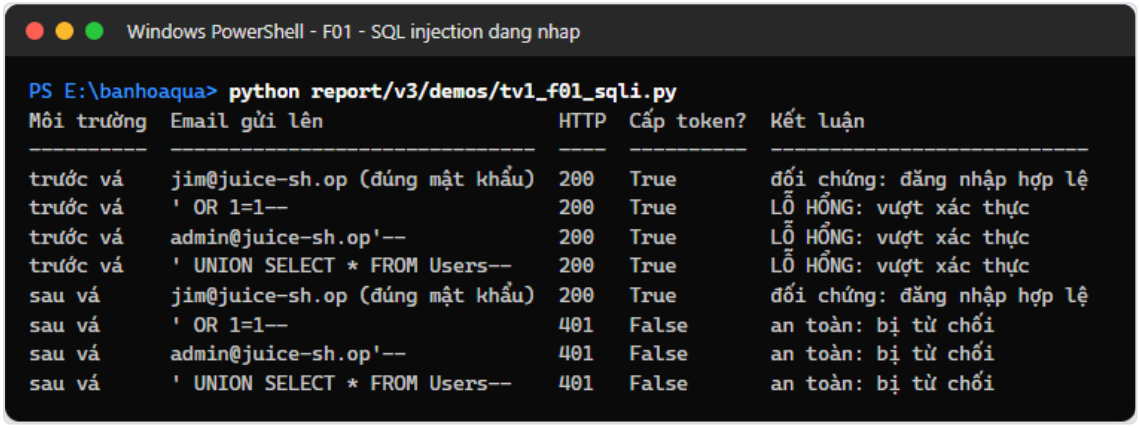
Sau khi sửa

Dùng truy vấn tham số (bind parameter):

```
const email = typeof req.body.email === 'string' ? req.body.email : ''
const password = typeof req.body.password === 'string' ? req.body.password : ''
models.sequelize.query('SELECT * FROM Users WHERE email = $email AND password = $password AND
deletedAt IS NULL',
  { bind: { email, password: security.hash(password) }, model: UserModel, plain: true })
```

Câu lệnh được biên dịch trước và giá trị truyền riêng, nên mọi ký tự SQL trong email giờ chỉ còn được hiểu là dữ liệu. Sau khi sửa, kiểm thử bảo mật xanh toàn bộ (Hình 13); demo cũng cho thấy mọi payload đều bị từ chối 401 trong khi đăng nhập hợp lệ vẫn hoạt động bình thường (Hình 11).

Bản vá vẫn còn một giới hạn: nó chỉ xử lý injection, trong khi cùng dòng code đó vẫn gọi `security.hash` của Juice Shop, vốn băm mật khẩu bằng **MD5 không salt** (`lib/insecurity.ts`, CWE-327 / OWASP A02:2021). Đây là lỗ hổng lưu trữ mật khẩu nằm ngoài phạm vi F01, vì F01 chỉ chặn việc vượt xác thực; khi triển khai thật, cần thay nó bằng bcrypt hoặc argon2 có salt.



Hình 11. Demo F01: cùng payload, trước khi sửa được cấp token, sau khi sửa bị từ chối

Chính sách vi phạm: GDPR Điều 32(1)(b) (bảo mật khi xử lý dữ liệu); Nghị định 13/2023 Điều 26 (biện pháp kỹ thuật bảo vệ dữ liệu cá nhân); Luật ATTTM 2015 Điều 19. **Pha DevSecOps:** Code → Test. **Kiểm soát (Security as Code):** phát hiện bằng SAST ở pha Code, chặn tái diễn bằng kiểm thử bảo mật tự động ở pha Test.

5.2. F02 — IDOR khi đọc giỏ hàng

Trước khi sửa

Route `routes/basket.ts` tìm giỏ chỉ theo mã trên URL:

```
const basket = await BasketModel.findOne({ where: { id }, include: [{ model: ProductModel, as:
'Products' }] })
```

Middleware chỉ xác nhận token hợp lệ chứ không kiểm tra quyền sở hữu, nên chỉ cần đổi `/rest/basket/8` thành `/rest/basket/9` là đọc được giỏ của người khác (CWE-639,

OWASP A01:2021).

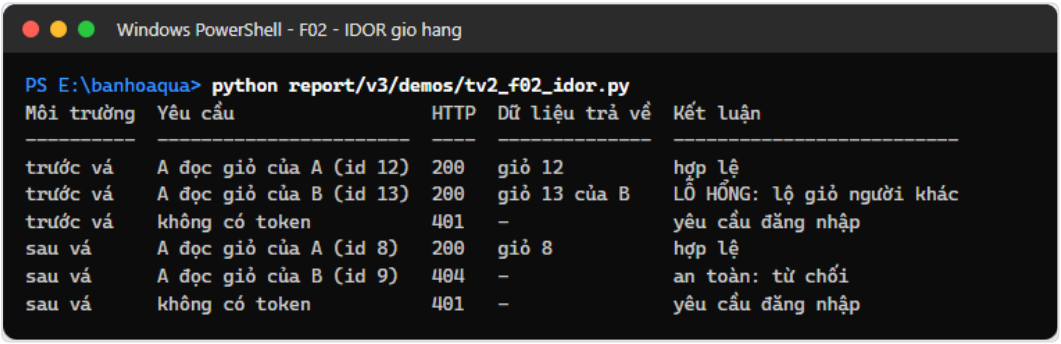
Phát hiện. Kịch bản kiểm thử dựng hai tài khoản A và B, rồi dùng token của A để đọc giỏ của B. Trên mã chưa sửa, máy chủ trả về đúng giỏ của B (Hình 12) và các ca tương ứng trong `f01-f02.test.ts` báo đỏ.

Sau khi sửa

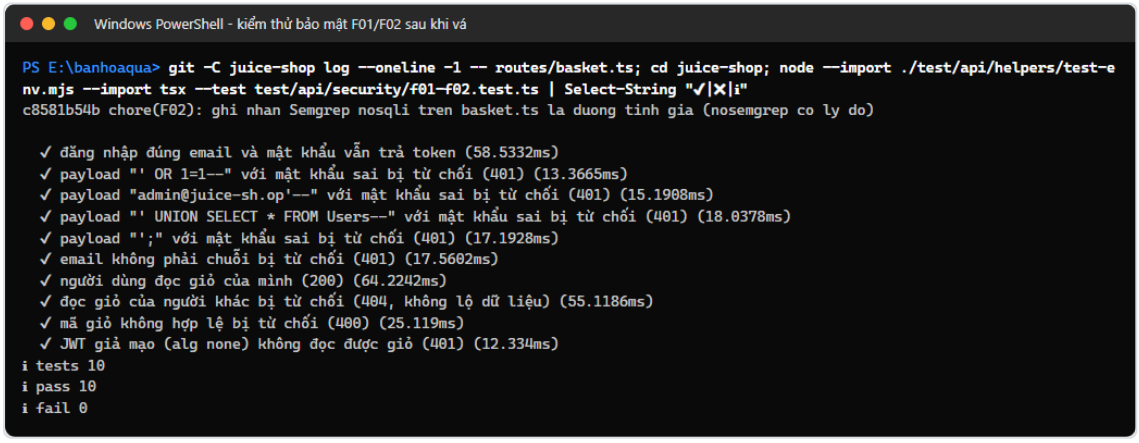
Giải pháp là ràng buộc chủ sở hữu ngay trong truy vấn, với danh tính lấy từ phiên do máy chủ cấp:

```
const user = security.authenticatedUsers.from(req)
if (!user?.data?.id) return res.status(401).end()
const id = Number(req.params.id)
if (!Number.isSafeInteger(id) || id <= 0) return res.status(400).end()
const basket = await BasketModel.findOne({ where: { id, UserId: user.data.id }, include: [{
model: ProductModel, as: 'Products' }] })
if (!basket) return res.status(404).end()
```

Sau khi sửa, A đọc giỏ của B chỉ nhận 404, còn giỏ của chính A vẫn trả 200 (Hình 12).



Hình 12. Demo F02: trước khi sửa A đọc được giỏ của B, sau khi sửa nhận 404



Hình 13. Kiểm thử bảo mật F01/F02 sau khi sửa: toàn bộ ca đạt

Chính sách vi phạm: GDPR Điều 5(1)(f), 25(2); Nghị định 13/2023 Điều 3 (nguyên tắc an toàn) và Điều 26; Luật ATTTM 2015 Điều 17 (không để lộ dữ liệu cá nhân cho bên thứ

ba). **Pha DevSecOps:** Design → Test. **Kiểm soát (Security as Code):** kiểm thử phân quyền tự động trong pipeline, chống tái diễn.

5.3. F03 — Lộ endpoint /metrics không xác thực

Trước khi sửa

Juice Shop mở sẵn `/metrics` (Prometheus) mà không yêu cầu đăng nhập, để lộ số người dùng, số đơn hàng và tổng số dư ví. Khi container publish thẳng cổng ứng dụng ra ngoài, bất kỳ ai cũng gọi được endpoint này từ host. Đây là một lỗi thuộc khâu **triển khai/vận hành**.

Phát hiện. Demo `demos/tv3_f03_metrics.py` gọi `/metrics` từ host; trước khi sửa, endpoint trả về 200 kèm hàng trăm dòng số liệu (Hình 14).

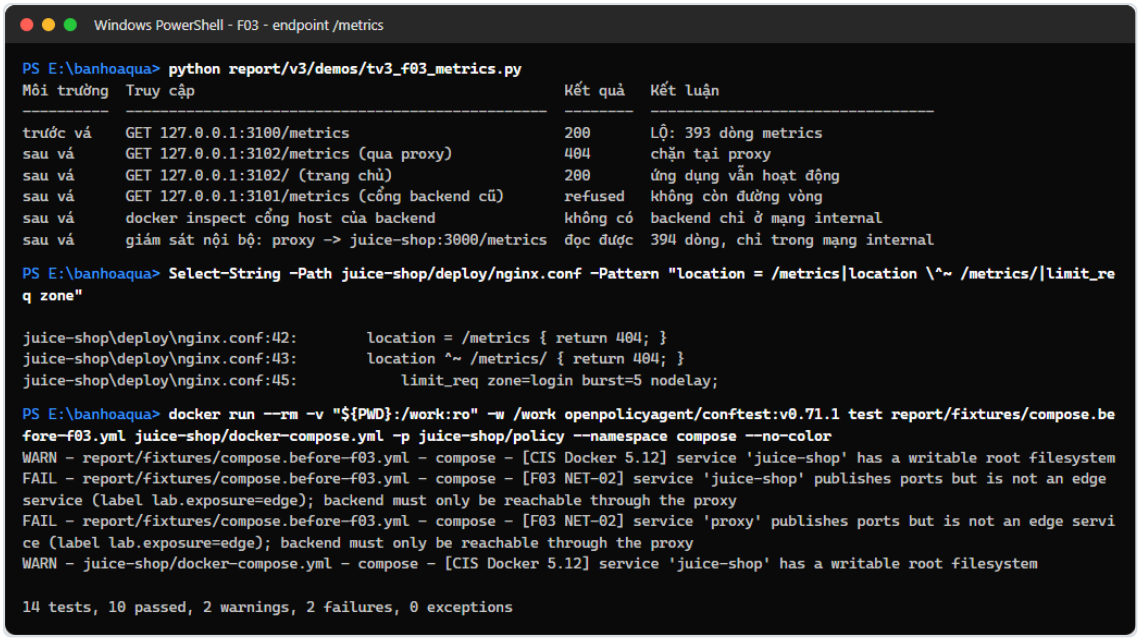
Sau khi sửa

Hướng xử lý là bỏ cổng host của backend và chỉ để reverse proxy mở cổng loopback; tại proxy, `/metrics` bị chặn và tốc độ đăng nhập bị giới hạn:

```
location = /metrics { return 404; }
location = /rest/user/login { limit_req zone=login burst=5 nodelay; limit_req_status 429;
proxy_pass http://juice-shop:3000; }
```

```
juice-shop:
  networks: [backend]            # không còn khóa ports -> không truy cập trực tiếp từ host
networks:
  backend: { internal: true }
```

Sau khi sửa, `/metrics` đi qua proxy trả về 404 và cổng backend cũ bị từ chối kết nối **từ host**. Đây là chặn ở lối vào từ bên ngoài chứ không phải khóa tuyệt đối: trong mạng internal, hệ thống giám sát vẫn đọc được metrics đúng mục đích (Hình 14).



Hình 14. Demo F03: /metrics lộ khi truy cập trực tiếp, bị chặn sau khi sửa

Chính sách vi phạm: GDPR Điều 25(2), 32(1)(b); Nghị định 13/2023 Điều 26; Luật ATTTM 2015 Điều 17, 19. **Pha DevSecOps:** Deploy → Operate. **Kiểm soát (Policy as Code):** luật `compose.rego` chặn mọi cấu hình để backend mở cổng host (mục 4).

5.4. F04 — Sai cấu hình an toàn của container

Trước khi sửa

Một image đóng gói Juice Shop có thể vi phạm chính sách an toàn tối thiểu mà vẫn lọt qua nếu kiểm tra đặt sai chỗ. `Dockerfile.vulnerable` dưới đây minh họa bốn vi phạm như vậy:

```
FROM node:22-alpine
COPY . .
ENV API_TOKEN=supersecret123  # nhúng bí mật vào image (lộ qua docker history)
EXPOSE 3000 22 3306           # mở cổng SSH 22, DB 3306
CMD ["npm", "start"]           # không USER (chạy root), không nhãn nguồn gốc
```

Những cách kiểm tra sơ sài rất dễ bị qua mặt. Kiểm tra kiểu "có lệnh `USER`" sẽ bỏ sót Dockerfile nhiều stage, bởi lệnh `USER` ở stage builder không áp dụng cho image chạy thật. Kiểm tra "`EXPOSE` không chứa cổng cấm" thì gần như vô nghĩa vì `EXPOSE` chỉ mang tính tài liệu. Còn nhãn `mfa="true"` thì ai cũng tự khai được mà chẳng phản ánh cấu hình thực tế.

Rủi ro nếu để nguyên, không vá. Mỗi vi phạm kéo theo một rủi ro an toàn cụ thể:

- **Chạy bằng root (LEAST-PRIVILEGE):** nếu ứng dụng bị chiếm quyền qua một lỗ hổng khác, chẳng hạn chính F01, tiến trình độc hại sẽ có ngay quyền root *bên trong* container. Khi đó kẻ tấn công dễ ghi đè file hệ thống, cài thêm công cụ, và nếu ghép

thêm một lỗ hổng nhân hệ điều hành thì có thể **thoát container và leo thang đặc quyền** lên máy chủ. Chạy non-root (UID 65532) giúp khoanh vùng thiệt hại ngay cả khi ứng dụng bị khai thác.

- **Mở cổng quản trị/CSDL (ATTACK-SURFACE):** khai báo EXPOSE 22 (SSH) và 3306 (MySQL) cho thấy các dịch vụ này có thể được bật, qua đó **mở rộng bề mặt tấn công** và tạo đường cho kẻ xấu dò quét, đăng nhập vét cạn vào dịch vụ quản trị và cơ sở dữ liệu, chính là nơi chứa dữ liệu người dùng.
- **Rủi ro chuỗi cung ứng (SUPPLY-CHAIN):** nếu thiếu nhãn nguồn gốc `org.opencontainers.image.source`/`revision`, khi có sự cố sẽ **không truy vết được image bắt nguồn từ commit nào** để điều tra hay thu hồi (đây là *provenance khai báo* theo SLSA L1, phục vụ truy vết; muốn chắc chắn hơn thì phải ký *attestation*). Thêm vào đó, dùng `:latest` hoặc chưa ghim digest khiến image có thể **bị đổi âm thầm** giữa các lần build, làm cho bản chạy thật khác với bản đã kiểm thử, còn lệnh `ADD` thì có thể kéo nội dung từ xa ngoài kiểm soát. Ghim digest, gắn nhãn nguồn gốc và dùng COPY sẽ giúp image **truy vết được, tái lập được và bất biến**.
- **Bí mật nhúng trong image (SECRET-IN-IMAGE):** đặt mật khẩu hay token trong `ENV`/`ARG` khiến bí mật **bị bake vào layer của ảnh**, và bất kỳ ai `docker pull` rồi chạy `docker history` đều đọc được, kể cả khi dòng đó đã bị xóa ở layer sau. Cách đúng là dùng *build secret* hoặc nạp biến môi trường lúc chạy. (Đây là **luật do nhóm bổ sung**, không có trong bản mẫu của lab.)

Sau khi sửa

Việc xử lý gồm **hai phần tách bạch**: (1) **sửa chính file triển khai** cho đúng cấu hình an toàn, và (2) **viết chính sách kiểm tra tự động** để chống tái diễn. Cũng cần nói rõ rằng **Conftest không tự sửa file**: nó chỉ đóng vai trò *cổng kiểm tra*, đọc Dockerfile rồi trả PASS/FAIL kèm lý do, và chặn phát hành nếu còn vi phạm. Người sửa thực sự vẫn là người viết Dockerfile.

(1) Sửa file Dockerfile triển khai. So với bản lỗi, bản vá (`report/fixtures/Dockerfile.remediated`) khắc phục đúng từng vi phạm tương ứng:

```
FROM node:22-alpine
# THÊM nhãn nguồn gốc OCI  -> đạt SUPPLY-CHAIN (provenance)
LABEL org.opencontainers.image.source="https://github.com/juice-shop/juice-shop" \
      org.opencontainers.image.revision="${VCS_REF}"
RUN addgroup -S -g 10001 juicers && adduser -S -u 10001 -G juicers juicers
COPY --chown=10001:0 . .
USER 10001:10001          # THÊM USER non-root       -> đạt LEAST-PRIVILEGE
EXPOSE 3000               # BỎ cổng 22 và 3306        -> đạt ATTACK-SURFACE
CMD ["npm", "start"]
```

Cụ thể, bản vá thêm lệnh `USER` để chạy non-root, loại hai cổng quản trị/CSDL khỏi `EXPOSE`, bổ sung nhãn nguồn gốc OCI và **không đặt bí mật trong** `ENV`/`ARG` (dùng *build secret* hoặc nạp biến môi trường lúc chạy). Khi phát hành thật, nên ghim base image theo digest, dù hiện luật Rego mới chỉ cảnh báo chứ chưa chặn. Image triển khai thực tế của nhóm (`deploy/Dockerfile.hotfix`) lẫn Dockerfile gốc của Juice Shop đều đã thỏa những yêu cầu này.

(2) Viết chính sách thành mã (Policy as Code) bằng Rego/Conftest (`juice-shop/policy/docker_security.rego`), với mỗi luật được đặt đúng điểm thực thi để không bị qua mặt. Mã nguồn đầy đủ của bộ luật nằm ở **Phụ lục A**.

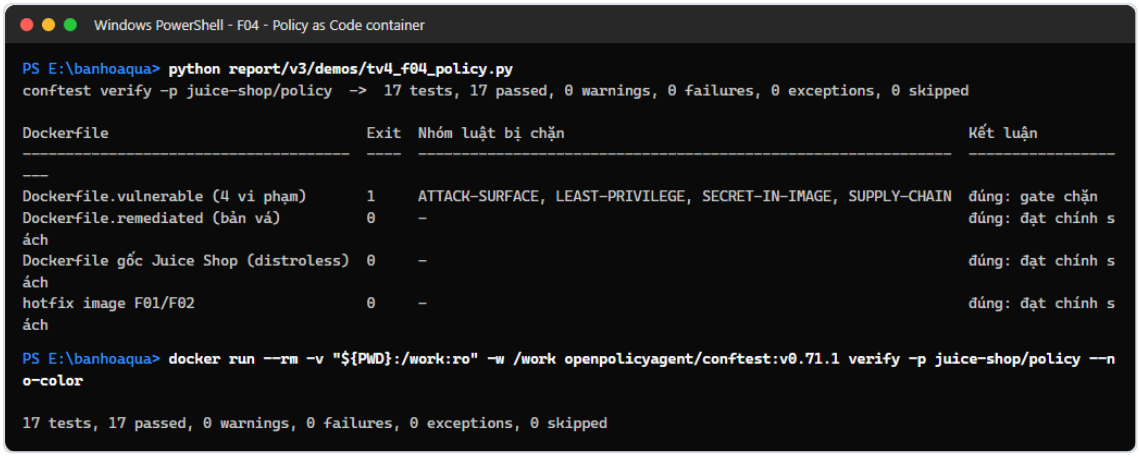
Cần lưu ý rằng Conftest là **bộ máy chính sách tổng quát và không có luật sẵn**, nên tự nó không quét ra lỗi nào. Chính **bộ luật Rego được viết riêng** (kèm mười một unit test chứng minh luật bắt đúng và không bị qua mặt) mới là *cách phát hiện* của F04. Đây cũng là cách Policy as Code vận hành: tổ chức tự mã hóa chính sách của mình, kiểm chứng và chạy lại được trong CI.

Các luật và tiêu chuẩn đối chiếu:

Bảng 10. Luật kiểm tra cấu hình container — luật yêu cầu gì và dựa trên chuẩn nào

Luật	Luật yêu cầu gì (chặn khi vi phạm)	Tiêu chuẩn (ý nghĩa xem Phụ lục C)
LEAST-PRIVILEGE	Stage cuối **phải** khai báo `USER` là UID số, không phải root	CIS Docker 4.1; PCI-DSS 2.2; NĐ 13/2023 Đ.26; Luật ATTTM Đ.19
ATTACK-SURFACE	**Không** được `EXPOSE` cổng quản trị/CSDL (22/23/3306/5432)	CIS Docker 5.8; NIST SP 800-190; Luật ATTTM Đ.19
SUPPLY-CHAIN	**Phải** có nhãn nguồn gốc OCI; **không** dùng `:latest`/`ADD`; nên ghim digest	OCI image-spec; SLSA; CIS Docker 4.5/4.9
SECRET-IN-IMAGE *(bổ sung)*	**Không** đặt secret/credential trong `ENV`/`ARG`	CIS Docker 4.10; PCI-DSS 8.2.1; GDPR Đ.32; NĐ 13/2023 Đ.26

Lệnh `conftest verify` chạy toàn bộ chính sách đạt 17/17 ca kiểm thử (11 ca cho chính sách Dockerfile của F04 và 6 ca cho chính sách compose của F03). Kết quả cho thấy `Dockerfile.vulnerable` bị chặn bởi 4 nhóm luật (LEAST-PRIVILEGE, ATTACK-SURFACE, SUPPLY-CHAIN, SECRET-IN-IMAGE), bản vá thì đạt, và Dockerfile gốc distroless của Juice Shop cũng **đạt toàn bộ chính sách an toàn** (Hình 15).



Hình 15. Demo F04: Dockerfile.vulnerable bị chặn bốn nhóm luật, bản vá và Dockerfile gốc đạt chính sách

Chính sách vi phạm: GDPR Điều 25(1), 32(1)(d); CIS Docker Benchmark 4.1; Nghị định 13/2023 Điều 26; Luật ATTTM 2015 Điều 19; TCVN 11930:2017. **Pha DevSecOps:** Build → Release. **Kiểm soát (Policy as Code):** bộ luật Rego chạy như một cổng kiểm tra bắt buộc trong pipeline.

6. Kết luận

Cả bốn lỗi an toàn (hai lỗi mã nguồn F01/F02 và hai lỗi triển khai F03/F04) đều được phát hiện bằng công cụ hoặc kiểm thử, rồi xử lý và kiểm chứng theo mạch trước→sau, kèm đoạn mã demo của từng thành viên. Toàn bộ quá trình phát hiện và chống tái diễn được thể hiện dưới dạng Policy as Code và Security as Code trong một chuỗi kiểm soát DevSecOps. Những biện pháp này vốn không có sẵn trong Juice Shop gốc, nơi lỗ hổng được để lại một cách cố ý, không có truy vấn tham số, không reverse proxy hay giới hạn tốc độ, không chính sách cấu hình container và cũng không có cổng kiểm soát nào trong CI. Đó chính là phần giá trị mà báo cáo bổ sung.

7. Tài liệu tham khảo

- OWASP Juice Shop — https://owasp.org/www-project-juice-shop/
- GDPR, Regulation (EU) 2016/679, Điều 5, 25, 32.
- Pháp luật Việt Nam: Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân (Điều 3, 26, 27); Luật An toàn thông tin mạng 2015 (86/2015/QH13, Điều 16, 17, 19); TCVN 11930:2017.
- CIS Docker Benchmark; PCI-DSS v3.2.1 (Yêu cầu 3.4, 8.2.1, 2.2); OWASP Top 10 2021; SLSA.
- Công cụ: Semgrep, Gitleaks, node:test (Supertest), Conftest/OPA.

- Tài liệu môn học: bộ slide DevSecOps và hướng dẫn Policy as Code (Conftest).

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Nguồn: kế thừa cấu trúc và các luật cốt lõi từ Appendix A.1 của tài liệu lab (`Policy_as_code_Lab ConfTest.docx`); phần nhóm điều chỉnh/bổ sung: bỏ luật HEALTHCHECK, gộp provenance vào SUPPLY-CHAIN, và **tự viết luật** **`SECRET-IN-IMAGE`** (xem mục 5.4). Namespace `dockerfile`, 8 luật `deny`; bộ kiểm thử `docker_security_test.rego` có 11 ca. Mỗi thông báo vi phạm trích kèm điều khoản chuẩn quốc tế và Việt Nam (NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19). Chạy: `conftest test <Dockerfile> -p juice-shop/policy --namespace dockerfile`.

```
# Policy as Code cho Dockerfile của Juice Shop — bám theo lab môn học "Policy as Code with
Conftest".
# Mỗi luật đặt tại điểm thực thi đúng thay vì các "original check" dễ bị qua mặt:
#   Least privilege  -> USER ở STAGE CUỐI, UID số, không phải root (thay vì "có USER bất kỳ trong
file")
#   Firewall/RBAC    -> cổng quản trị/DB không được EXPOSE (giảm bề mặt tấn công)
#   Supply chain     -> provenance khai báo (nhãn OCI source+revision, SLSA L1) + không :latest,
không ADD, cảnh báo base chưa ghim digest
#   Secret hygiene   -> KHÔNG nhúng bí mật (ENV/ARG) vào image (cải tiến của nhóm, ngoài bản mẫu
lab)
# Mỗi thông báo vi phạm trích sẵn điều khoản chuẩn áp dụng — quốc tế (CIS Docker, PCI-DSS, NIST,
OCI/SLSA)
# và Việt Nam (Nghị định 13/2023/NĐ-CP Điều 26 — biện pháp kỹ thuật bảo vệ dữ liệu cá nhân; Luật
An toàn
# thông tin mạng 2015 Điều 19 — biện pháp bảo đảm an toàn thông tin cá nhân, tuân thủ tiêu chuẩn
kỹ thuật).
#   chạy: conftest test <Dockerfile> -p policy/ --namespace dockerfile
package dockerfile

import rego.v1

# Danh sách cổng quản trị/CSDL — giữ theo Appendix A.1 của lab (SSH, telnet, Docker daemon,
MySQL, Postgres, Redis, Mongo).
forbidden_ports := {"22", "23", "2375", "2376", "3306", "5432", "6379", "27017"}

required_labels := {"org.opencontainers.image.source", "org.opencontainers.image.revision"}

root_names := {"root", "0", "root:root", "0:0"}

# Chỉ STAGE CUỐI tạo ra image chạy thật; USER ở stage builder không áp dụng cho runtime.
final_stage := max([ins.Stage | some ins in input])

final_users := [ins.Value[0] | some ins in input; ins.Stage == final_stage; ins.Cmd == "user"]

last_user := final_users[count(final_users) - 1] if count(final_users) > 0

# LEAST-PRIVILEGE ------------------------------------------------------------
deny contains msg if {
        count(final_users) == 0
        msg := "LEAST-PRIVILEGE: final stage has no USER instruction; container runs as root.
(CIS Docker 4.1; PCI-DSS 2.2; NIST SP 800-190; NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19)"
}

deny contains msg if {
```

```
        last_user in root_names
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' is root. (CIS Docker 4.1; PCI-DSS 2.2;
NĐ 13/2023 Đ.26)", [last_user])
}

warn contains msg if {
        last_user
        not last_user in root_names
        not regex.match(`^[0-9]+(:[0-9]+)?$`, last_user)
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' is not a fixed numeric UID. (CIS Docker
4.1)", [last_user])
}

# ATTACK-SURFACE (firewall / RBAC) ----------------------------------------
port_number(raw) := split(raw, "/")[0] # bỏ hậu tố /tcp, /udp

deny contains msg if {
        some ins in input
        ins.Cmd == "expose"
        some raw in ins.Value
        port_number(raw) in forbidden_ports
        msg := sprintf("ATTACK-SURFACE: EXPOSE '%s' covers administrative/database port %s. (CIS
Docker 5.8; NIST SP 800-190; Luật ATTTM 2015 Đ.19)", [raw, port_number(raw)])
}

# SUPPLY-CHAIN — provenance khai báo (traceability; không thay cho chữ ký/attestation) --
label_blob := concat(" ", [concat(" ", ins.Value) | some ins in input; ins.Cmd == "label"])

deny contains msg if {
        some req in required_labels
        not contains(label_blob, req)
        msg := sprintf("SUPPLY-CHAIN: missing provenance label '%s' (declared provenance for
traceability). (OCI image-spec; SLSA Provenance L1; NIST SSDF)", [req])
}

# SUPPLY-CHAIN — base image (không :latest/ADD, ghim digest) -----------------
deny contains msg if {
        some ins in input
        ins.Cmd == "from"
        endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' uses :latest; pin a version or digest. (CIS
Docker 4.5; SLSA)", [ins.Value[0]])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "add"
        msg := "SUPPLY-CHAIN: use COPY instead of ADD to avoid remote/implicit extraction. (CIS
Docker 4.9)"
}

warn contains msg if {
        some ins in input
        ins.Cmd == "from"
        not contains(ins.Value[0], "@sha256:")
        not endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' is not pinned by digest. (CIS Docker 4.5;
SLSA)", [ins.Value[0]])
}

# SECRET-IN-IMAGE — cải tiến của nhóm (ngoài bản mẫu lab) --------------------
# Bí mật đặt trong ENV/ARG bị bake vào layer ảnh và lộ qua `docker history`. Ánh xạ
# luật secret của compose (F03) xuống tầng Dockerfile. ENV parse thành [key, value, "=", ...];
# ARG parse thành ["KEY=VALUE"].
secret_key := `(?i)(password|passwd|secret|token|api[_-]?key|access[_-]?key|private[_-]?
```

```
key|credential)`

deny contains msg if {
        some ins in input
        ins.Cmd == "env"
        some i, key in ins.Value
        i % 3 == 0
        i + 1 < count(ins.Value)
        regex.match(secret_key, key)
        ins.Value[i + 1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ENV '%s' hardcodes a credential into the image (exposed
via docker history); use build secrets/runtime env. (CIS Docker 4.10; GDPR Art.32; PCI-DSS 8.2.1;
NĐ 13/2023 Đ.26)", [key])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "arg"
        some tok in ins.Value
        contains(tok, "=")
        regex.match(secret_key, split(tok, "=")[0])
        split(tok, "=")[1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ARG '%s' bakes a credential default into the image. (CIS
Docker 4.10; GDPR Art.32; PCI-DSS 8.2.1; NĐ 13/2023 Đ.26)", [split(tok, "=")[0]])
}
```

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Nguồn: nhóm tự viết cho docker-compose — lab chỉ enforce trên manifest Kubernetes, nhóm chuyển các ý tưởng least-privilege/firewall/secret sang định dạng triển khai thật. Namespace `compose`, 5 luật `deny` (chặn backend mở cổng host, bind ngoài loopback, privileged, mount docker.sock, secret nội tuyến); bộ kiểm thử `compose_test.rego` có 6 ca. Thông báo vi phạm trích kèm điều khoản chuẩn quốc tế và Việt Nam (NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19). Chạy: `conftest test docker-compose.yml -p juice-shop/policy --namespace compose`.

```
package compose

import rego.v1

# Input: file docker-compose (conftest tự đọc YAML).
# ports chấp nhận cả 3 dạng của Compose: "127.0.0.1:3102:8080", 3000, {target, published,
host_ip}.
# Thông báo vi phạm trích điều khoản chuẩn quốc tế (GDPR, PCI-DSS, CIS Docker) và Việt Nam
# (Nghị định 13/2023/NĐ-CP Điều 26; Luật An toàn thông tin mạng 2015 Điều 19).

host_ip(p) := split(p, ":")[0] if {
        is_string(p)
        count(split(p, ":")) == 3
}

host_ip(p) := "" if {
        is_string(p)
        count(split(p, ":")) < 3
}

host_ip(p) := "" if is_number(p)

host_ip(p) := object.get(p, "host_ip", "") if is_object(p)

ports(svc) := object.get(svc, "ports", [])

# Dịch vụ được thiết kế để nhận kết nối từ host (reverse proxy, dashboard) phải tự khai báo nhãn.
edge(svc) if object.get(object.get(svc, "labels", {}), "lab.exposure", "") == "edge"

# F03 / NET-02: backend không được publish cổng. Nếu backend có cổng host, mọi route (kể cả
/metrics)
# có thể bị truy cập vòng qua proxy, làm vô hiệu luật chặn /metrics và giới hạn tốc độ ở proxy.
deny contains msg if {
        some name, svc in input.services
        count(ports(svc)) > 0
        not edge(svc)
        msg := sprintf("[F03 NET-02] service '%s' publishes ports but is not an edge service
(label lab.exposure=edge); backend must only be reachable through the proxy (Luật ATTTM 2015
Đ.19; NĐ 13/2023 Đ.26)", [name])
}

# NET-01 (GDPR Art.32): cổng publish chỉ được bind loopback trong lab.
deny contains msg if {
        some name, svc in input.services
```

```
        some p in ports(svc)
        host_ip(p) != "127.0.0.1"
        msg := sprintf("[GDPR Art.32 NET-01] service '%s' publishes %v on all interfaces (NĐ
13/2023 Đ.26; Luật ATTTM 2015 Đ.19)", [name, p])
}

deny contains msg if {
        some name, svc in input.services
        svc.privileged == true
        msg := sprintf("[CIS Docker 5.4] service '%s' runs privileged", [name])
}

deny contains msg if {
        some name, svc in input.services
        some v in object.get(svc, "volumes", [])
        contains(sprintf("%v", [v]), "docker.sock")
        msg := sprintf("[CIS Docker 5.31] service '%s' mounts the Docker socket", [name])
}

# Biến môi trường có thể là danh sách "KEY=value" hoặc map {KEY: value}; *_FILE trỏ tới secret
thì được phép.
env(svc) := [e | some e in svc.environment] if is_array(svc.environment)

env(svc) := [sprintf("%s=%v", [k, v]) | some k, v in svc.environment] if
is_object(svc.environment)

env(svc) := [] if not svc.environment

deny contains msg if {
        some name, svc in input.services
        some e in env(svc)
        regex.match(`(?i)^[A-Z0-9_]*(PASSWORD|SECRET|TOKEN|API_KEY)=.+`, e)
        msg := sprintf("[PCI-DSS 8.2.1 / GDPR Art.32 / NĐ 13/2023 Đ.26] service '%s' has an
inline secret in environment: %s", [name, split(e, "=")[0]])
}

drops_all_caps(svc) if "ALL" in object.get(svc, "cap_drop", [])

warn contains msg if {
        some name, svc in input.services
        not drops_all_caps(svc)
        msg := sprintf("[CIS Docker 5.3] service '%s' does not drop capabilities", [name])
}

warn contains msg if {
        some name, svc in input.services
        not svc.read_only
        msg := sprintf("[CIS Docker 5.12] service '%s' has a writable root filesystem", [name])
}
```

Phụ lục C — Giải thích tiêu chuẩn và điều khoản tham chiếu

Bảng tra ý nghĩa (**yêu cầu gì**) của mọi tiêu chuẩn/điều khoản được trích trong báo cáo. Phần thân chỉ ghi mã; ý nghĩa tra tại đây.

Bảng 11. Tra cứu — mỗi tiêu chuẩn/điều khoản tham chiếu yêu cầu gì

Điều khoản	Yêu cầu / ý nghĩa tóm gọn
GDPR Điều 5(1)(f)	Nguyên tắc *toàn vẹn & bảo mật*: xử lý dữ liệu một cách an toàn
GDPR Điều 25	Bảo vệ dữ liệu *từ thiết kế* (khoản 1) và *theo mặc định* (khoản 2)
GDPR Điều 32	Biện pháp kỹ thuật & tổ chức bảo đảm an toàn khi xử lý — (1)(b) bí mật/toàn vẹn, (1)(d) kiểm thử & đánh giá định kỳ
OWASP A01:2021 / CWE-639	*Broken Access Control* — kiểm soát truy cập bị phá vỡ (IDOR)
OWASP A02:2021 / CWE-327	*Cryptographic Failures* — lỗi mật mã (vd băm mật khẩu bằng thuật toán yếu)
OWASP A03:2021 / CWE-89	*Injection* — tiêm nhiễm dữ liệu vào câu lệnh (SQL injection)
PCI-DSS 2.2	Cấu hình tham số an toàn hệ thống, chạy với đặc quyền tối thiểu
PCI-DSS 3.4	Làm số thẻ (PAN) không đọc được ở mọi nơi lưu trữ
PCI-DSS 8.2.1	Không lưu thông tin xác thực (credential) ở dạng đọc được
CIS Docker 4.1	Chạy container bằng user riêng, **không phải root**
CIS Docker 4.5	Dùng ảnh nền tin cậy, ghim phiên bản/digest (tránh tag động `:latest`)
CIS Docker 4.9	Dùng `COPY` thay cho `ADD` (tránh kéo nội dung từ xa ngoài kiểm soát)
CIS Docker 4.10	Không lưu bí mật trong Dockerfile
CIS Docker 5.3	Loại bỏ Linux capabilities không cần (`cap_drop: ALL`)
CIS Docker 5.4	Không chạy container ở chế độ `privileged`
CIS Docker 5.8	Chỉ mở đúng cổng cần thiết (không mở cổng quản trị/CSDL)
CIS Docker 5.12	Gắn hệ thống tệp gốc ở chế độ chỉ đọc (`read_only`)
CIS Docker 5.31	Không mount Docker socket vào container
OCI image-spec	Chuẩn định dạng ảnh container + nhãn metadata nguồn gốc
SLSA (L1)	Khung toàn vẹn chuỗi cung ứng; mức L1 = *có khai báo nguồn gốc (provenance)*
NIST SP 800-190	Hướng dẫn an toàn cho container
NĐ 13/2023 Điều 3	Nguyên tắc bảo vệ dữ liệu cá nhân (gồm nguyên tắc bảo mật, an toàn)
NĐ 13/2023 Điều 26	Biện pháp quản lý & kỹ thuật bảo vệ dữ liệu cá nhân, từ đầu và suốt quá trình xử lý
NĐ 13/2023 Điều 27	Biện pháp bảo vệ dữ liệu cá nhân cơ bản

Điều khoản	Yêu cầu / ý nghĩa tóm gọn
Luật ATTTM 2015 Điều 16	Nguyên tắc bảo vệ thông tin cá nhân
Luật ATTTM 2015 Điều 17	Không chia sẻ thông tin cá nhân cho bên thứ ba khi chưa được đồng ý
Luật ATTTM 2015 Điều 19	Biện pháp quản lý & kỹ thuật bảo đảm an toàn thông tin cá nhân; tuân thủ tiêu chuẩn kỹ thuật
TCVN 11930:2017	Yêu cầu cơ bản về an toàn hệ thống thông tin theo cấp độ